

Relatório Aula 4

- Felipe Uehara Gondo (13680612)
- Gabriel Enrico Tomimori (14746401)

Vídeo

- [Encoder - PSI3422](#)

Código

```
#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/gpio.h>
#include <zephyr/sys/atomic.h>
#include <stdio.h>

#define FAIXAS_PRETAS 24
#define TEMPO_AMOSTRAGEM_MS 1000

// Captura o pino PTA1
#define ENCODER_NODE DT_ALIAS(encoder)
static const struct gpio_dt_spec encoder_pin = GPIO_DT_SPEC_GET(ENCODER_NODE,
gpios);

static struct gpio_callback encoder_cb_data;

// atomic_t garante segurança quando duas interrupções acessam a mesma variável
static atomic_t contador_pulsos = ATOMIC_INIT(0);
static float velocidade_rpm = 0.0;

// Interrupção do GPIO: chamada quando o sensor detecta a faixa preta
void encoder_isr(const struct device *dev, struct gpio_callback *cb, uint32_t
pins) {
    atomic_inc(&contador_pulsos);
}

// Interrupção do Timer: chamada a cada 100ms
void calcula_rpm_timer_handler(struct k_timer *dummy) {
    // Lê o valor atual e zera o contador em uma única operação segura
    atomic_val_t pulsos = atomic_set(&contador_pulsos, 0);

    // rpm = (pulsos / faixas) * (60000 / tempo_ms)
    velocidade_rpm = ((float)pulsos / FAIXAS_PRETAS) * (60000.0 /
TEMPO_AMOSTRAGEM_MS);
}

// Define o timer no kernel do Zephyr
K_TIMER_DEFINE(rpm_timer, calcula_rpm_timer_handler, NULL);

int main(void) {
    // Verifica se o dispositivo (pino) está pronto no hardware
    if (!gpio_is_ready_dt(&encoder_pin)) {
        return 0;
    }

    // Configura o pino como entrada
    gpio_pin_configure_dt(&encoder_pin, GPIO_INPUT);

    // Configura a interrupção para borda de subida (HIGH)
```

```
gpio_pin_interrupt_configure_dt(&encoder_pin, GPIO_INT_EDGE_RISING);

// Associa a função "encoder_isr" ao pino
gpio_init_callback(&encoder_cb_data, encoder_isr, BIT(encoder_pin.pin));
gpio_add_callback(encoder_pin.port, &encoder_cb_data);

// Inicia o timer: dispara a primeira vez em 100ms e repete a cada 100ms
k_timer_start(&rpm_timer, K_MSEC(TEMPO_AMOSTRAGEM_MS),
K_MSEC(TEMPO_AMOSTRAGEM_MS));

while (1) {
    printf("Velocidade: %.2f RPM\n", velocidade_rpm);
    k_msleep(500); // Aguarda meio segundo antes de imprimir novamente
}

return 0;
}
.....
```